



RehearsalHub

Dokumentace k ročníkové práci

Autor: Andrej Zdvořák

Třída: 3ITB

Vedoucí práce: Bc. Vratislav Medřický

2025/2026

Prohlášení

Prohlašuji, že jsem ročníkovou práci na téma „RehearsalHub“ vypracoval samostatně a s použitím uvedené literatury a pramenů.

V Opatně dne 05.06.2026

.....

Poděkování

Rád bych poděkoval Bc. Vratislavovi Medřickému za vedení mé ročníkové práce, cenné rady a odborný dohled. Děkuji Bc. Jiřímu Mítovi a Bc. Jiřímu Brožovi za pomoc při grafické úpravě a obsahové kontrole práce. Mé poděkování patří mé matce za jazykovou a gramatickou korekturu. Děkuji také některým spolužákům za pomoc s testováním a za jejich návrhy.

Anotace

Předkládaná dokumentace popisuje kompletní proces návrhu a realizace mobilní aplikace RehearsalHub, která slouží pro organizaci a komunikaci hudebních kapel. Text je rozdělen do dvou hlavních částí. Teoretická část se zabývá rešerší konkurenčních řešení, jako jsou *BandHelper* či *Mattermost*, a detailně rozebírá zvolené technologie, včetně frameworku *React Native*, platformy *Expo* a backendového řešení postaveného na *Node.js* a *PostgreSQL*. Praktická část dokumentuje postup vývoje od prvotních skic a návrhu databázového modelu (ERD) přes tvorbu UI designu ve *Figmě* až po programování klíčových funkcí, jako je autentizace, správa kapel a implementace real-time chatu. Závěr práce obsahuje uživatelskou příručku a zhodnocení splnění stanovených cílů.

Klíčová slova

Kapela, organizace, nápad, zkouška, koncert, spolupráce, komunikace, nahrávka, úkol, setlist, aplikace

Obsah

Úvod	8
1 Teoretická část	9
1.1 Rešerše	9
1.1.1 BandHelper	9
1.1.2 Mattermost	10
1.2 Technologie	11
1.2.1 React Native + Expo	11
1.2.2 Node.js	11
1.2.3 NativeWind	11
1.2.4 PostgreSQL + PgAdmin	11
1.2.5 Firebase	12
1.2.6 Figma	12
1.2.7 GitHub	12
1.2.8 ChatGPT	12
2 Praktická část	13
2.1 Návrhy	13
2.1.1 Hrubý náčrt struktury	13
2.1.2 ERD databáze	14
2.1.3 Základní návrh designu aplikace	15
2.2 Produktizace	16
2.2.1 Uživatelé	16
2.2.2 Kapely	18
2.2.3 Skupinový chat kapely	19
2.2.4 Stránka uživatelského profilu	22
2.2.5 Úložiště skladeb a audio systém	23
2.2.6 Události	25
2.2.7 Správa úkolů (Todos)	26
2.2.8 Sdílení hudebních nápadů	26
2.2.9 Individuální záznam cvičení	28
2.2.10 Setlisty	29
2.2.11 Domovská obrazovka (Dashboard)	30
2.3 Popis pro uživatele	31
2.3.1 Registrace a prvotní nastavení	31
2.3.2 Vytvoření nebo připojení ke kapele	31

2.3.3	Nástěnka a přehled aktivit.....	32
2.3.4	Plánování událostí.....	32
2.3.5	Správa repertoáru a přehrávač.....	33
2.3.6	Komunikace	33
2.3.7	Správa úkolů	34
2.3.8	Měření cvičení	34
2.3.9	Nahrávání a editace hudebních nápadů.....	34
2.3.10	Tvorba setlistů	34
Závěr		35
Seznam použitých zdrojů.....		36
Seznam obrázků		37
Seznam kódů		37

Úvod

Hlavním cílem ročníkové práce je vytvořit mobilní aplikaci pro menší kapely, která na jednom místě propojí plánování zkoušek a koncertů, práci se skladbami a setlisty¹, sdílení hudebních nápadů a základní týmovou komunikaci. Toto téma jsem si vybral, protože propojuje moje dlouhodobé zájmy – hudbu a vývoj softwaru – a současně se zaměřuje na reálný problém, který se u mnoha kapel bez manažera v praxi objevuje. Tím problémem je roztržitost nástrojů (chat, kalendář, cloud, nápady, poznámky), které minimálně i menší kapela potřebuje používat. Protože jsou tyto nástroje každý v jiné aplikaci a typicky nejsou nijak propojené, se organizace a komunikace stávají obtížnější, náročnější, a hlavně ztrácí přehlednost. Moje aplikace má toto roztržitění eliminovat a nabídnout kapelám přehledné prostředí přizpůsobené jejich běžným potřebám. Primárně cílím na menší a nezávislé kapely bez dedikovaného manažera, návrh ale nechává prostor k rozšíření i pro větší sestavy.

Aplikace nabídne kapelám nástroje jako plánování akcí s možností upozornění, tvorbu setlistů opakovaně použitelných u různých koncertů, nahrání a okamžité sdílení hudebního nápadu s ostatními členy kapely, integrovaný skupinový chat kapely a úložiště skladeb s jejich stavem rozpracovanosti.

Aby bylo možné na konci projekt objektivně zhodnotit, je potřeba stanovit měřitelné cíle. Za úspěch projektu budu považovat, pokud bude k dispozici sestavení aplikace pro Android a iOS a uživatel v ní zvládne: zaregistrovat se nebo se přihlásit (buď pomocí emailu a hesla nebo přihlášení přes Google), založit kapelu nebo se připojit do již existující kapely pomocí pozvánkového kódu, ukládat a spravovat skladby kapely s volitelnými metadaty a stavem rozpracovanosti, používat skupinový chat pro komunikaci v kapele, plánovat zkoušky a koncerty a následně zobrazit jejich seznam, sestavit setlist a přiřadit ho ke konkrétnímu koncertu, uložit hudební nápad a sdílet ho pro ostatní členy, vytvářet si vlastní seznam úkolů s možností vedoucího kapely přiřadit úkol a individuálně zaznamenávat údaje o cvičení. Aplikace bude respektovat systémové nastavení (světlý/tmavý režim a velikost písma), bude použitelná i na atypických poměrech stran a bude obsahovat podmínky používání přímo v aplikaci.

Očekávám, že mi projekt přinese praktické zkušenosti s návrhem a vývojem mobilních aplikací. Od návrhu uživatelského rozhraní za použití UX standardů přes práci s externím úložištěm a autentizací až po návrh databázového modelu a tvorbu backendu. Rámcově plánuji použít multiplatformní přístup vývoje a běžně používaný soubor technologií, důraz však budu klást i na UX doporučení platforem a základní zásady přístupnosti (např. čitelnost, ovladatelnost, práce s dynamickou velikostí textu). V průběhu vývoje budu průběžně testovat funkce na typických scénářích, také s dalšími osobami, aby byla ověřena funkčnost výsledného řešení, které by mělo být užitečné i mimo školní prostředí.

Rozsah a hranice první verze volím s ohledem na časové možnosti, cíle práce a náročnost řešení. V projektu plánuji používat technologie, které zatím neovládám, proto jsou hranice trochu užší. Měl jsem například nápad na funkci pro oddělování jednotlivých stop skladeb pro zjednodušení cvičení, gamifikaci cvičení/aplikace pomocí achievementů, dostal jsem také návrh implementovat do aplikace fankluby kapel, zpětnou vazbu fanoušků a soukromé zprávy mezi jednotlivci. Tyto funkce jsou momentálně příliš časově a strukturálně náročné na to, aby byly implementovány v rámci ročníkového projektu, ale po jeho ukončení bych se těmito funkcím chtěl věnovat. Aktuální prioritou je spolehlivá funkčnost, přehlednost a propojení. V závěru se vrátím k uvedeným cílům a vyhodnotím jejich naplnění.

¹ Setlist - seznam skladeb, které umělec nebo kapela plánuje zahrát během živého vystoupení

1 Teoretická část

Tato kapitola se věnuje teoretickému pozadí projektu. Cílem je přiblížit existující řešení a principy, ze kterých projekt vychází, zasadit jej do širšího kontextu, a ukázat jeho inspiraci a využívané technologie.

1.1 Rešerše

Rešerše slouží k analýze existujících aplikací a služeb s podobným zaměřením. Jejím cílem je identifikovat běžné funkce, silné a slabé stránky a inspirovat se pro návrh vlastního řešení.

1.1.1 BandHelper

BandHelper je cloudová služba s doprovodnými mobilními aplikacemi, která je navržena tak, aby snížila organizační zátěž spojenou s hraním v kapele. Slouží jako úložiště pro veškerý repertoár, rozvrh, kontakty a finanční informace a bez námahy je sdílí s kolegy z kapely. Je to robustní systém, který se kromě organizace kapely zaměřuje i na použití během živého vystoupení, například pro automatizaci osvětlení. (1)

Z pohledu mého projektu je však BandHelper ukázkou toho, jak může být aplikace pro začátečníky příliš složitá ("feature creep"). Uživatelské rozhraní je plné tabulek a složitého nastavení, což může odradit kapely, které se chtějí jen "domluvit a hrát", navíc je design aplikace na dnešní standardy zastaralý a uživatelsky nepřívětivý. RehearsalHub se vůči tomuto vymezuje – mým cílem není nabídnout více funkcí než BandHelper, ale nabídnout ty nejdůležitější (chat, soubory, kalendář) v moderním a přehledném vzhledu, který nevyžaduje hodiny nastavování.

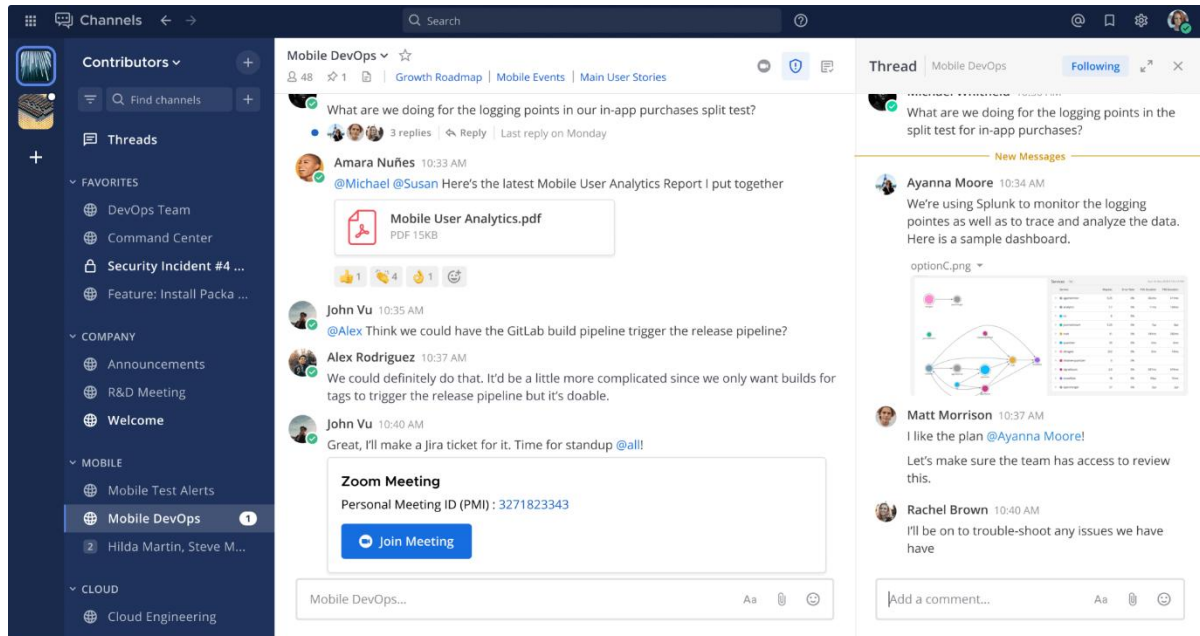


Obrázek 1 - Ukázka vzhledu aplikace BandHelper. Zdroj: (2)

1.1.2 Mattermost

To, co mě k nápadu na tento projekt nejvíce inspirovalo, byla aplikace Mattermost, na kterou jsem narazil v rámci mé brigády – aplikace pro komunikaci a organizaci ve firmách. (3)

Mattermost nemá nic společného s hudbou, ale má podobný cíl jako RehearsalHub. Sjednocuje funkce jako kalendář, chat a úkoly na jedno místo.



Obrázek 2 - Mattermost. Zdroj: (3)

1.2 Technologie

1.2.1 React Native + Expo

React Native je open-source framework vyvinutý společností Meta, který umožňuje tvorbu nativních mobilních aplikací pomocí jazyka JavaScript nebo TypeScript. Právě TypeScript jsem si pro tento projekt zvolil díky jeho typové kontrole, která pomáhá předcházet chybám v kódu. Hlavní výhodou této technologie je možnost sdílet jeden kódový základ mezi platformami Android a iOS, což výrazně urychluje vývoj oproti psaní dvou oddělených nativních aplikací. (4)

Expo je sada nástrojů a služeb postavená nad React Native, která zjednodušuje vývojový proces. Odstraňuje nutnost složité konfigurace nativních souborů a poskytuje aplikaci Expo Go. Díky ní mohu okamžitě testovat změny v aplikaci fyzicky přímo na svém telefonu pouhým naskenováním QR kódu, což je pro rychlé prototypování v rámci ročníkové práce klíčové. (5)

1.2.2 Node.js

Node.js je běhové prostředí (runtime environment) postavené na V8 JavaScript enginu prohlížeče Chrome, které umožňuje spouštět JavaScript na straně serveru. V mém projektu slouží jako základ backendu, kde zpracovává požadavky z mobilní aplikace a komunikuje s databází. (6)

Tuto technologii jsem vybral proto, že mi umožňuje používat stejný programovací jazyk (TypeScript) jak na frontendu (mobilní aplikace), tak na backendu (server). *Node.js* využívám společně s nástrojem NPM (Node Package Manager), který mi zpřístupňuje rozsáhlou knihovnu předpřipravených balíčků a modulů. Díky tomu nemusím programovat vše od nuly, ale mohu integrovat ověřená řešení pro autentizaci nebo práci se soubory.

1.2.3 NativeWind

NativeWind je knihovna, která umožňuje používat systém stylů Tailwind CSS přímo v prostředí React Native. Zatímco v běžném vývoji mobilních aplikací se styly píšou jako JavaScriptové objekty, NativeWind mi dovoluje psát styly přímo do atributu *className* pomocí zkrácených tříd (např. *flex-1*, *bg-black*, *text-white*). (7)

Tuto technologii jsem zvolil především kvůli rychlosti vývoje a přehlednosti kódu. Díky tomu, že znám Tailwind CSS z vývoje webových stránek, mohu uživatelské rozhraní stylovat mnohem efektivněji, než kdybych musel definovat každý styl zvlášť. NativeWind navíc při kompilaci převádí tyto třídy na nativní styly, takže nedochází ke snížení výkonu aplikace. (7)

1.2.4 PostgreSQL + PgAdmin

PostgreSQL je pokročilý open-source relační databázový systém, který je známý svou stabilitou, robustností a dodržováním SQL standardů. V mém projektu slouží jako hlavní úložiště strukturovaných dat – ukládám zde informace o uživatelích, kapelách, setlistech a jejich vzájemných vazbách (např. informace o tom, jakou roli má konkrétní uživatel v dané kapele). Relační model je pro aplikaci typu RehearsalHub klíčový, protože data jsou silně provázaná. (8)

Pro správu databáze využívám nástroj PgAdmin. Jedná se o grafické rozhraní, které mi umožňuje vizuálně navrhovat tabulky, spouštět SQL dotazy a kontrolovat data uložená na serveru, aniž bych musel vše řešit pouze přes příkazový řádek. To mi výrazně usnadňuje ladění backendu a kontrolu správnosti ukládaných údajů.

1.2.5 Firebase

Firebase je komplexní platforma od společnosti Google pro vývoj mobilních a webových aplikací. Ačkoliv pro logiku a vztahy mezi daty používám PostgreSQL, Firebase v projektu využívám primárně pro jeho službu Cloud Storage a Authentication. (9)

Jelikož aplikace pracuje s multimediálními soubory – konkrétně s nahrávkami nápadů, tabulaturami v PDF nebo soubory z DAW – potřebuji spolehlivé a škálovatelné úložiště pro tyto binární soubory. Ukládat velké soubory přímo do databáze PostgreSQL by nebylo efektivní. Firebase mi umožňuje tyto soubory nahrát do cloudu a v databázi uchovávat pouze odkaz (URL) na daný soubor, což je standardní a výkonné řešení. Dále mi umožňuje přihlásit uživatele přes externí služby jako je Google a zároveň i běžně přes email a heslo. (9)

1.2.6 Figma

Figma je vektorový grafický editor a nástroj pro tvorbu prototypů, který funguje přímo ve webovém prohlížeči. Ještě před napsáním prvního řádku kódu jsem tento nástroj využil pro kompletní návrh vizuální podoby aplikace (UI) a uživatelské přívětivosti (UX). (10)

Vytvořil jsem zde design jednotlivých obrazovek, jako je dashboard, chat nebo detail skladby, a navrhl jsem, jak na sebe budou navazovat. Díky tomu jsem si mohl ujasnit rozložení prvků a barevnou paletu, což mi následně ušetřilo čas při samotném programování, protože jsem přesně věděl, jak má výsledek vypadat.

1.2.7 GitHub

GitHub je webová služba pro hosting softwarových projektů, která využívá verzovací systém Git. Pro můj projekt je naprosto nezbytná z hlediska zálohování a správy historie kódu. (11)

Díky GitHubu mám kód bezpečně uložený v cloudu a mohu se kdykoliv vrátit k předchozím funkčním verzím aplikace, pokud by při vývoji došlo k chybě. Slouží mi také jako portfolio mé práce a umožňuje mi přehledně sledovat postup vývoje pomocí tzv. commitů, kde popisuji, co jsem v dané části práce změnil nebo přidal. (11)

1.2.8 ChatGPT

ChatGPT je jazykový model vyvinutý společností OpenAI, založený na architektuře GPT. V rámci své ročníkové práce jej využívám jako "inteligentního asistenta" a konzultanta. (12)

Neslouží mi k tomu, aby psal kód za mě, ale pomáhá mi s vysvětlením dokumentace k novým knihovnám, hledáním chyb v kódu (debugging) nebo s optimalizací složitějších algoritmů. Často jej využívám také pro generování testovacích dat do databáze, abych mohl aplikaci testovat na reálném objemu informací. ChatGPT používám jako můj hlavní zdroj učení se s pro mě novými technologiemi.

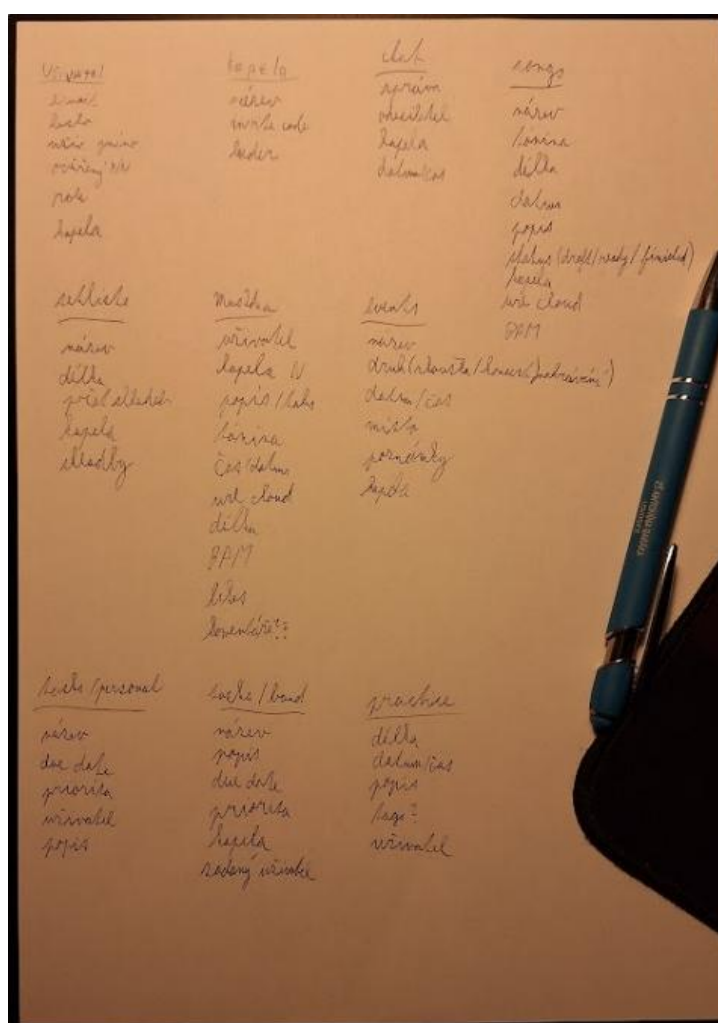
2 Praktická část

Praktická část popisuje samotný návrh a realizaci aplikace. Zaměřuje se na proces vzniku projektu od prvotního nápadu až po technickou implementaci.

2.1 Návrhy

2.1.1 Hrubý náčrt struktury

Než jsem se pustil do dalších návrhů, potřeboval jsem si nejdřív ujasnit myšlenky kolem samotného nápadu. Udělal jsem to tak, že jsem si na papír začal zapisovat jednotlivé oblasti aplikace a všechno, co s nimi souvisí nebo by mělo být uloženo v databázi. Byl to úplně první, spíš prvotní návrh, který se v průběhu času hodně změnil, ale pomohl mi udělat si základní přehled a posunout se dál.

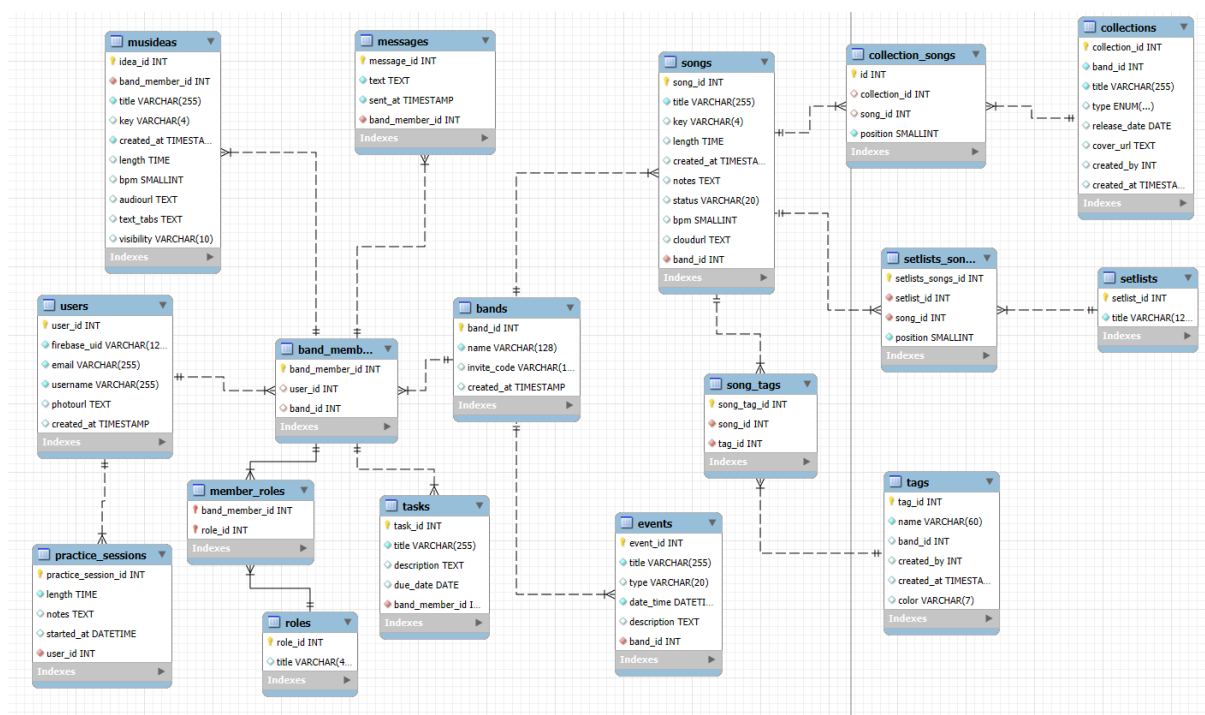


Obrázek 3 - Fotografie papíru s prvním návrhem projektu. Zdroj: autor

2.1.2 ERD databáze

Poté co jsem více koncept aplikace promyslel, jsem udělal návrh databáze v MySQL Workbench. První ERD bylo o dost menší než finální, protože mě až během vývoje napadlo více věcí, např. jsem až v průběhu prosince přidával tabulky jako kolekce a štítky vztahující se ke skladbám, z důvodu, že by bylo dobré mít nějakou možnost, jak skladby filtrovat a organizovat.

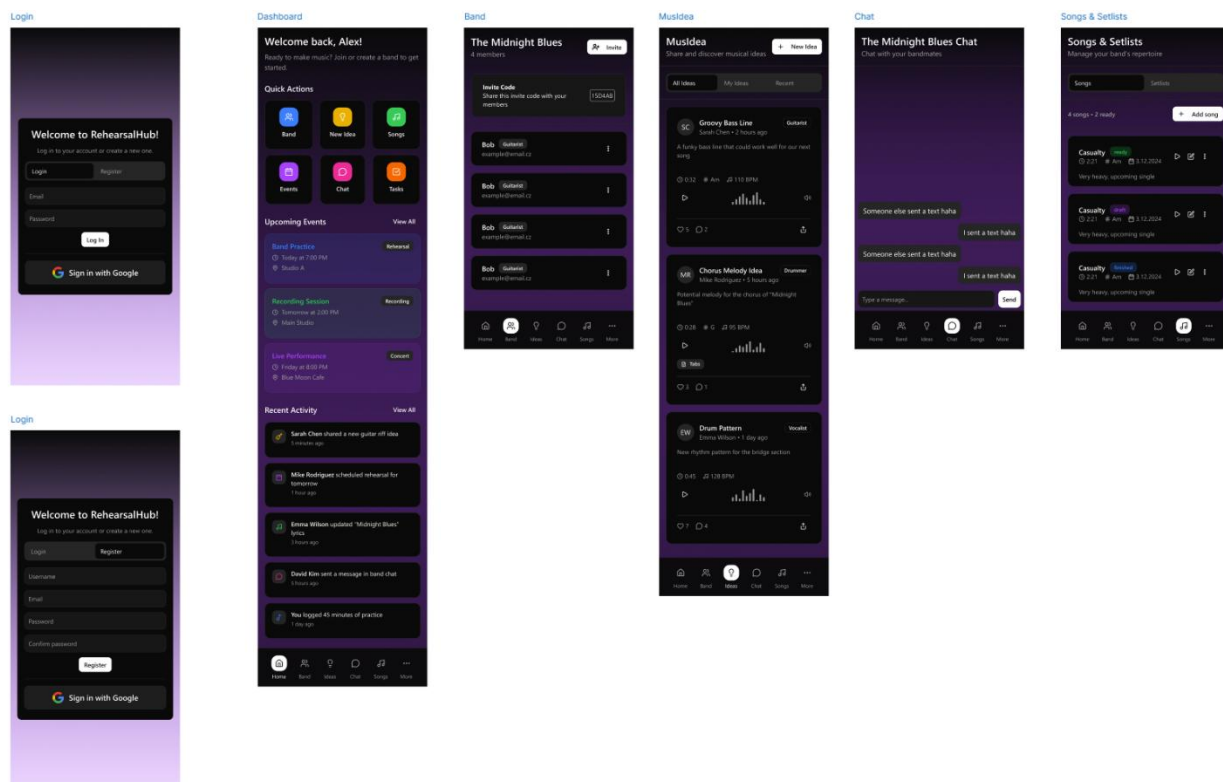
Nejdůležitějšími tabulkami jsou tabulky *band_members* a *bands*. Tabulka *band_members* je spojovací tabulkou mezi uživateli a kapelami, obsahuje ID uživatele a ID kapely, jsou na ni napojeny tabulky s nápady, rolemi, zprávami a úkoly. Kromě toho se na ni často dotazuje moje API. Tabulka *bands* uchovává informace o kapele, zastřešuje tabulky pro skladby a události, a je navržena tak, aby umožňovala snadné napojení dalších funkcí v budoucnu.



Obrázek 4 - ERD Databáze. Zdroj: autor

2.1.3 Základní návrh designu aplikace

Pro designování jsem zvolil nástroj Figma. Než jsem začal designovat, už jsem měl v hlavě představu, jak by aplikace měla vypadat – minimalisticky a především přehledně. Když jsem otevřel Figma, tak jsem si všiml nové funkce, která díky umělé inteligenci umí navrhnout design přesně podle toho, co jí řeknete. Rozhodl jsem se tuto funkci vyzkoušet a umělá inteligence mi poskytla opravdu dobrý výsledek téměř podle toho, jak jsem si to představoval. Část tohoto návrhu jsem pak převedl na klasický *Figma Design* projekt a upravil ho tak, aby návrh odpovídal tomu, co jsem chtěl. Nedělal jsem návrh úplně každé stránky aplikace, udělal jsem dostatek stránek k tomu, abych design zbylých stránek mohl odhadnout až během vývoje v kódu.



Obrázek 5 - Návrh designu aplikace. Zdroj: autor

2.2 Produktizace

2.2.1 Uživatelé

První, co jsem musel udělat, abych mohl pokračovat ve všem ostatním byla registrace a přihlášení uživatele. Na návrh vedoucího práce jsem k tomu použil Firebase Auth, který nejen že spravuje přihlášení sám, ale také dává možnost přihlášení přes externí služby. Já jsem se rozhodl využít registraci pomocí emailu a hesla, a přihlášení pomocí Googlu. S tím jsem měl ale problém, protože nastavení přihlašování přes Google bylo komplikované a dlouho se mi nedařilo zprovoznit. Problém byl v tom, že tato možnost nefunguje v aplikaci Expo Go, ve které jsem přihlášení testoval, takže jsem musel udělat sestavení a v *Google Cloud Console* jsem musel povolit možnost používat vlastní URI², i přesto, že tato možnost byla označena jako nedoporučená.

Firebase Auth ale uchovává pouze email, unikátní identifikátor a heslo, což nestačilo pro moje potřeby, protože jsem potřeboval uchovávat profilový obrázek uživatele a jeho uživatelské jméno. Proto jsem musel vytvořit tabulku *users*, pomocí funkcí Firebase získat vygenerovaný unikátní identifikátor a ten uložit do databázové tabulky společně s profilovým obrázkem (pokud proběhla registrace přes Google) a uživatelským jménem. K uložení do databáze jsem použil volání svého API endpointu, který před vložením záznamu zkontroluje, zda již emailová adresa, identifikátor nebo uživatelské jméno není používáno a případně vrátí chybovou hlášku, která je následně zobrazena uživateli.

Požadavkem projektu bylo mimo jiné ověření emailové adresy, což bylo nezbytné vzhledem k tomu, že jsem dal uživateli možnost se pomocí ní registrovat. To mi opět usnadnil *Firebase*, který má funkci pro zaslání ověřovacího odkazu pomocí konfigurovatelného emailu. Takže poté, co uživatel zadá své registrační údaje a klikne na tlačítko *Register* ho aplikace odkáže na stránku, kde si může znovu nechat zaslat ověřovací email nebo kliknout na tlačítko *Verified*, pokud už se ověřil. S ověřovacím emailem jsem ale během testování se spolužákem zjistil, že email vůbec nepříjde na doménu *seznam.cz*, kdežto s doménou *gmail.com* nebyl žádný problém. Každý emailový server má totiž svůj filtr, který nepustí dál emaily z určitých emailových serverů. Proto jsem musel email nakonfigurovat na emailovou adresu s mojí vlastní doménou, u které jsem musel nastavit správně DNS, a nastavit používání emailového serveru této domény. Po této změně už vše fungovalo správně.

² URI – standardní řetězec znaků, který jednoznačně identifikuje digitální zdroj (webovou stránku, obrázek, dokument, email, aplikaci) v počítačové síti

Uživatel je v aplikaci velmi klíčový, na každé stránce se na něj musím dotazovat a ověřovat, zda je v kapele, co má za kapely a tak dále. Proto jsem využil možnosti vytváření kontextů. Kontext obaluje celou aplikaci (můžu si ovšem zvolit, jestli obaluje i další kontexty), tudíž je přístupný ze všech stránek a může uchovávat data a může definovat funkce. Proto jsem vytvořil kontext *AuthContext*, který si uchovává všechny informace o uživateli a zároveň má funkce, které jsou potřebné k uživatelské správě, tj. funkce pro registraci, přihlášení, přihlášení pomocí Googlu a odhlášení.

```
// Definice typu uživatele
export type AppUser = {
  uid: string;
  email: string | null;
  username: string | null;
  photoURL: string | null;
  emailVerified: boolean;
};

// definice typu kontextu
type AuthContextType = {
  user: AppUser | null;
  loading: boolean;
  idToken: string | null;
  setIdToken: (idToken: string | null) => void;
  register: (
    email: string,
    password: string,
    username: string
  ) => Promise<void>;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
  googleSignIn: (firebaseUser?: FirebaseUser) => Promise<void>;
  updateUser: (data: Partial<AppUser>) => void;
};

// vytvoření kontextu
const AuthContext = createContext<AuthContextType | null>(null);
export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("UseAuth must be inside AuthProvider");
  return ctx;
};
```

Kód 1 - Definice autentizačního kontextu

2.2.2 Kapely

Kapela je bezpochybně neklíčovější částí mé aplikace. Váže se na ni téměř vše ostatní – skladby, setlisty, chat, členové a nápady. Než jsem ale začal aplikaci vyvíjet, musel jsem se zamyslet nad tím, zda může být uživatel ve vícero kapelách, neboť jsem si myslel, že v reálném světě se toto téměř neděje. Jenže po průzkumu jsem zjistil, že je to naopak docela časté, zejména pro studiové hudebníky³. Při návrhu databáze jsem tedy spojil tabulky pro uživatele a kapely spojovací tabulkou, abych vytvořil kardinalitu M:N. Jak to ale řešit v kódu aplikace? Zde jsem opět použil kontext, který uchovává data o kapele, na která se na určitých stránkách dotazuji. Kontext jsem vybavil funkcí pro změnu kontextu kapely, kterou využívám v menu na každé stránce a na stránce profilu – tam lze nalézt přepínač mezi kapelami, kterými je uživatel součástí.

```
type BandContextType = {
  bands: Band[];
  activeBand: Band | null;
  bandsLoading: boolean;
  switchBand: (id: string) => Band | null;
  createBand: (name: string, roles: BandRole[]) => Promise<void>;
  joinBandByCode: (code: string, role: BandRole[]) => Promise<void>;
  fetchUserBands: (uid: string) => Promise<void>;
  fetchBandMembers: (
    bandId: string
  ) => Promise<{ members: any[]; currentUserRoles: string[] }>;
  removeBand: (bandId: string) => void;
  removeBandMember: (bandId: string, firebaseUid: string) => Promise<void>;
  makeLeader: (bandId: string, newLeaderFirebaseUid: string) =>
    Promise<void>;
};
// tvorba kontextu kapely
const BandContext = createContext<BandContextType | undefined>(undefined);
export const useBand = () => {
  const context = useContext(BandContext);
  if (!context) {
    throw new Error("useBand has to be called inside BandProvider");
  }
  return context;
};
```

Kód 2 - Definice kontextu kapely

Nejprve se musí ale kapela vytvořit. K tomu jsem definoval další funkci v kontextu kapel – do té vstupuje jméno kapely a role, které si zakládající uživatel vybral. Funkce použije volání API endpointu, který vytvoří náhodný kód pro připojení do kapely, vloží záznam do tabulky *band*, spojovací tabulky *band_members*, najde ID rolí z tabulky *roles* a ty vloží společně s ID uživatele do tabulky *member_roles*. Kromě toho automaticky zakládajícímu uživateli přiřadí roli *Leader*, protože se dá předpokládat, že zakládající uživatel je jakýmsi vedoucím kapely. Tato role slouží ke kontrole oprávnění, například změnu

³ Studiový hudebník – profesionální hudebník najímaný k nahrávání nástrojových partů, který není stálým členem kapely

jména kapely smí udělat pouze vedoucí. Role vedoucího lze předat jinému členovi, a je to vynucené v případě, že chce vedoucí kapelu opustit.

Pro připojení do kapely slouží již zmíněný generovaný unikátní kód. Uživatel ho může zkopírovat a poslat jeho kolegům, kteří tento kód zadají v sekci *join band*, kde si opět mohou zvolit několik z předpřipravených rolí. Pro připojení slouží další funkce v kontextu, která funguje obdobně jako funkce pro vytvoření kapely, jen do ní nevstupuje jméno ale pozvánkový kód. API vkládá záznamy o rolích a do spojovací tabulky mezi kapelami a uživateli.

2.2.3 Skupinový chat kapely

Chat byl asi doposud nejsložitější částí aplikace, co se týče vývoje, zabral nejvíce času a ani tak není dokonalý. Při plánování projektu jsem se rozhodl, že chat nechám jednoduchý – do chatu nelze přidávat soubory ani obrázky a každá kapela má pouze jeden skupinový chat, tzn. nelze chatovat jeden na jednoho. Tento návrh byl zvolen záměrně s cílem zachovat přehlednost a zabránit roztříštění informací do více konverzací. Architektura aplikace ovšem umožňuje budoucí rozšíření o tuto funkcionalitu, která ale není součástí mého plánu.

Jak jsem již zmínil, chat byl technicky nejnáročnější částí aplikace. Nechtěl jsem, aby uživatelé museli neustále obnovovat stránku, aby viděli nové zprávy (tzv. polling), proto jsem se rozhodl pro real-time komunikaci. K tomu jsem využil knihovnu *Socket.IO*, která zajišťuje obousměrné spojení mezi serverem a klientem pomocí WebSockets⁴.

Na straně backendu jsem využil konceptu „místností“ (rooms). Jakmile uživatel otevře chat, klient pošle požadavek na připojení do místnosti *band:ID_KAPELY*. Tím je zajištěno, že zprávy se doručí pouze členům dané kapely a nikomu jinému. Autentizace socketu probíhá stejně jako u zbytku aplikace – při navázání spojení se odesílá *Firebase* ID token, který server ověří a identifikuje uživatele.

Zajímavou výzvou bylo načítání historie zpráv a jejich zobrazování. Použil jsem komponentu *FlatList* s vlastností *inverted={true}*. To v praxi znamená, že seznam je technicky „vzhůru nohama“ – první položka v poli je renderována na spodku obrazovky. Tento přístup má dvě obrovské výhody:

1. Při otevření chatu je uživatel automaticky na konci konverzace (u nejnovějších zpráv) bez nutnosti scrollovat.
2. Když uživatel posune prstem nahoru a načtou se starší zprávy, seznam „neodskočí“, protože se nové položky přidávají na „konec“ (technicky na konec pole), což se vizuálně projeví nahore.

⁴ Websockets – komunikační protokol, který umožňuje obousměrnou komunikaci v reálném čase (full-duplex) mezi klientem a serverem

Načítání starších zpráv neprobíhá přes WebSockets, ale klasickým REST API voláním, aby se nezatěžovalo socketové spojení přenosem velkých dat. Využil jsem zde tzv. kurzorovou paginaci (cursor pagination). Klient si pamatuje datum odeslání nejstarší načtené zprávy a při požadavku na další historii pošle tento časový údaj serveru. Server pak vrátí nastavený počet zpráv starších než tento datum a posune kurzor.

```
const getMessageHistory = async (loadOlder = false) => {
  // zkráceno pro dokumentaci
  const params = new URLSearchParams();
  if (loadOlder && nextCursor) params.set("before", nextCursor);
  params.set("limit", "40");

  const token = await auth.currentUser?.getIdToken();
  const res = await fetch(
    `${apiUrl}/api/messages/${activeBand.id}?${params}`,
    { headers: { Authorization: `Bearer ${token}` } }
  );

  const { items, nextCursor: newCursor } = await res.json();

  setMessages(prev =>
    loadOlder ? [ prev, items ] : items
  );
  setNextCursor(newCursor ?? null);
};
```

Kód 3 - Načítání historie a kurzorová paginace

Aby byl chat pocitově rychlý (tzv. Optimistic UI⁵), nečekám při odeslání zprávy na potvrzení serveru. Zprávu ihned vypíšu do seznamu se stavem *pending* (označeno ikonou načítání). Teprve až server potvrdí přijetí a uložení do databáze, změní se stav na „ok“. Pokud by server neodpověděl nebo vrátil chybu (např. kvůli výpadku internetu), zpráva zčervená a umožní uživateli opakované odeslání.

```
const handleMessageSend = (text string) => {
  // zkráceno pro dokumentaci
  const tempId = Date.now().toString();
  // optimistic message
  setMessages(prev => [{
    clientId: tempId,
    text,
    status: "pending",
    sent_at: new Date().toISOString(),
  }, ...prev]);

  socket.emit("message:send", { text, tempId }, res => {
    setMessages(prev =>
      prev.map(m =>
        m.clientId === tempId
          ? res.ok
            ? { ...res.message, status: "ok" }
            : { ...m, status: "failed" }
          : m
        )
      )
    );
  });
};
```

Kód 4 – Logika odeslání zprávy (Optimistic Update)

⁵ Optimistic UI – technika návrhu a vývoje, kde aplikace okamžitě aktualizuje UI (zobrazí výsledek akce uživatele), aniž by čekala na potvrzení ze serveru, a předpokládá, že operace proběhne úspěšně

2.2.4 Stránka uživatelského profilu

Stránka profilu je rozdělená na tři části: část profilu, kde se lze odhlásit a upravit svůj profil; změna barevného schématu a změna kontextu kapely.

Při upravování profilu jsou dvě možnosti: změna profilového obrázku a změna uživatelského jména. Při registraci pomocí emailu a hesla na začátku uživatel samozřejmě nemůže mít žádný profilový obrázek, proto jsem ve své databázi nastavil omezení *DEFAULT*, které automaticky atribut profilového obrázku vyplní adresou základního obrázku. Při jeho změně se nahraje nový do složky uživatele na Firebase Storage a automaticky se ořízne na poměr stran 1:1. Změna uživatelského jména je provedena jednoduchým přepsáním záznamu v databázi pomocí mého API.

Se změnou barevného schématu mi velice pomohla knihovna NativeWind. Ta má totiž speciální stylovou třídu *dark*:, která specifikuje, které třídy se mají používat, v případě že je nastavený tmavý režim. To znamená, že při tvorbě UI jsem u každé komponenty psal třídy jak pro světlý, tak pro tmavý režim. Ovšem pro komponenty, které nedisponují atributem stylových tříd, jsem musel použít React Native hook ⁶ *useColorScheme()*, jež jsem použil v podmínce pro zjištění aktuálního barevného schématu. Tohle vše reagovalo na systémové nastavení schématu, avšak já jsem chtěl tlačítko pro změnu mezi světlým, tmavým nebo systémovým schématem. Proto jsem musel vytvořit kontext *ThemeContext*, který jsem vybavil funkcemi pro nastavení nebo změnu schématu. Informace o jeho nastavení je ukládána do zařízení, což umožňuje zachovat správné schéma i po zavření a opětovném otevření aplikace. Kromě toho jsem musel upravit soubor hooku *useColorScheme()*, aby používal obdobný hook z knihovny NativeWind. *ThemeContext* obaluje celou aplikaci, a tak se díky hooku *useEffect* načte barevné schéma automaticky.

```
useEffect(() => {
  const loadTheme = async () => {
    try {
      const saved = await AsyncStorage.getItem("user-theme-preference");
      if (
        saved === "light" || saved === "dark" || saved === "system"
      ) {
        setThemeState(saved);
        setColorScheme(saved);
      } else {
        setColorScheme("system");
      }
    } catch (error) {
      console.log("Failed to load theme preference", error);
    } finally {
      setLoaded(true);
    }
  };
  loadTheme();
}, []);
```

Kód 5 - Načtení barevného schéma z *ThemeContextu*

⁶ Hook – je speciální funkce, která umožňuje funkčním komponentám pracovat se stavem, životním cyklem a další logikou aplikace (např. reakce na změny dat) bez použití tříd.

Změna kontextu kapely je provedena jednoduchým *select* polem, který ve svém dropdownu zobrazuje všechny kapely uživatele. Při kliknutí na jednu z kapel se zavolá funkce pro přepnutí kapely z mého *BandContextu*. Toto pole pro změnu kontextu kapely jsem přidal do rozbalovacího menu na každé stránce.

2.2.5 Úložiště skladeb a audio systém

Úložiště skladeb je další komplexní částí aplikace. Umožňuje kapele ukládat skladby ze svého repertoáru na cloudové úložiště, a tak jsou přístupné odkudkoliv.

Při vývoji jsem začal formulářem pro přidání skladby, který po uživateli vyžaduje vlastnosti jako název, délka, tempo, tóninu a status rozpracování. Poté jsem se přesunul k samotné logice nahrání jakéhokoliv audio souboru na cloudové úložiště Firebase Storage. Celý proces probíhá v těchto krocích:

1. Backend zkontroluje, zda má kapela v úložišti svou složku (případně ji vytvoří) a vygeneruje tzv. podepsanou adresu pro bezpečné nahrání.
2. Frontend tuto adresu získá a pomocí funkce *uploadAsync()* z *FileSystem* knihovny *Expo* zahájí nahrávání souboru.
3. Po úspěšném nahrání souboru se do databáze uloží záznam s informacemi o skladbě a cestou k jejímu souboru.

Knihovnu *FileSystem* jsem zvolil především proto, že její funkce *uploadAsync()* umožňuje sledovat průběh nahrávání, což bylo nezbytné pro vytvoření načítací lišty. Bez ní by uživatel nebyl informován o průběhu akce. Lišta vizuálně reprezentuje dvě události: samotný upload souboru a následné uložení záznamu do databáze. Protože u API požadavku pro uložení do databáze nelze měřit průběh, navrhl jsem chování tak, že se lišta po dokončení uploadu zastaví na 95 %. Zbýlých 5 % se postupně doplňuje v intervalu, dokud nepřijde potvrzení. Toto chování jsem implementoval pro případ pomalého připojení, aby aplikace nepůsobila „zamrzlým“ dojmem.

```
async function uploadFileToSignedUrl({ localUri, contentType, bandId,
onProgress }) { // Zkráceno pro dokumentaci
  // 1. Získání Signed URL
  const res = await fetch(`${apiUrl}/api/songs/upload-url`, {
    method: "POST", body: JSON.stringify({filename, contentType, bandId})
  });
  const { uploadUrl } = await res.json();
  // 2. Konfigurace Upload Tasku (Expo FileSystem)
  const task = FileSystem.createUploadTask(uploadUrl, localUri,
    { httpMethod: "PUT", uploadType: 0, headers: { "Content-Type:
contentType } },
    (data) => {
      const p = (data.totalBytesSent / data.totalBytesExpectedToSend) * 100
      onProgress(Math.min(Math.round(p), 95)); // Zastavení na 95 %
    });
  // 3. Spuštění nahrávání
  const result = await task.uploadAsync();
  if (result?.status === 200) onProgress(100); // Hotovo
}
```

Kód 6 - nahrávání souboru na cloudové úložiště

Po vyvinutí této základní funkčnosti mě ale napadlo, že by si uživatel určitě přál mít možnost skladby nějak filtrovat, řadit a organizovat – něco, na co jsem během plánování vůbec nepomyslel. Vymyslel jsem tedy dvě nové věci: filtrování pomocí štítků a zařazování skladeb do kolekcí. Vytvořil jsem tedy nové tabulky *tags* a *collections* a k nim spojovací tabulky k *songs*. Do tabulky *tags* jsem vložil několik základních štítků a ty jsem zobrazil ke zvolení ve formuláři. Kromě toho jsem dal uživateli možnost přidat vlastní štítky přímo pod pevně vloženými štítky.

Na stránku skladeb jsem nakonec přidal vyhledávací pole, menu pro seřazení skladeb a okno pro filtraci podle tří různých vlastností: status skladby, tónina skladby a štítky skladby. Filtrace je nastavena tak, že při výběru více možností v rámci jednoho filtru se zobrazí skladby, které splňují alespoň jednu z vybraných podmínek, nikoli pouze ty, které splňují všechny zároveň.

Poslední, co chybělo k dokončení stránky se skladbami, byla možnost skladby přehrávat. Zde jsem se rozhodl použít design plovoucí lišty vykreslené nad celou aplikací, je tedy přístupná na každé stránce. Myslel jsem na to, že později budu přidávat stránku pro ukládání nápadů, kde bude také potřeba soubory přehrávat. Proto jsem vytvořil nový kontext *AudioPlayerContext*, který využívá knihovnu *expo-audio*. V něm jsem vytvořil typ *PlayableAudio*, který má společné vlastnosti jak skladeb, tak nápadů. Samotný kontext obsahuje funkce pro manipulaci s audiem. Knihovna *expo-audio* umožňuje vytvořit přehrávač audia pomocí funkce, do které vstupuje adresa souboru audia. Pomocí příslušných funkcí se následně přehrávač ovládá. Vytvořil jsem také komponentu malé lišty, která zobrazuje informace o skladbě, možnost ukončit poslech, pauzu a spuštění a posuvnou lištu pro posouvání času skladby. Lištu skladby jsem schoval na stránce chatu, protože překážela výhledu.

```
type PlayableAudio = {
  song_id: number;
  title: string;
  url: string;
  type: "song" | "idea";
};
type PlayerState = {
  current: PlayableAudio | null;
  isPlaying: boolean;
  play: (song: PlayableAudio) => void;
  pause: () => void;
  stop: () => void;
  resume: () => void;
  player: any;
  clearCurrent: () => void; // pro ukončení přehrávání
};
```

Kód 7 - Definice kontextu přehrávače

2.2.6 Události

Plánování je pro kapelu stejně důležité jako samotné hraní, proto jsem sekci událostí věnoval větší pozornost. Mým cílem bylo vytvořit přehledný systém, kde kapela okamžitě vidí, co ji čeká, a co už má za sebou.

Při návrhu databáze a logiky jsem se rozhodl rozdělit události do dvou základních typů: Zkouška (Rehearsal) a Koncert (Concert). Každý typ má svá specifika – zatímco u koncertu je klíčové místo konání a délka vystoupení, u zkoušky je důležitější vědět, co se bude hrát. Proto jsem ve formuláři pro vytvoření události, který jsem stejně jako všechny ostatní postavil na knihovně *Formik* v kombinaci s validačním schématem *Yup*, implementoval dynamické pole. Pokud uživatel zvolí typ „Zkouška“, aplikace mu nabídne seznam skladeb z repertoáru kapely, které chce na zkoušce cvičit. Tyto skladby se pak uloží do vazební tabulky *event_songs*.

Na frontendu jsem se zaměřil na přehlednost. Události jsem rozdělil do dvou záložek – nadcházející a minulé, mezi kterými uživatel přepíná pomocí mojí komponenty *SwitchTabs*. Pro rychlou vizuální orientaci jsem každému typu události přiřadil specifickou barvu (zelená pro zkoušky, modrá pro koncerty), která se promítá do pozadí karet i textů.

U koncertů jsem zatím nechal připravené místo pro setlisty. Ačkoliv databázová struktura je již navržena, samotné propojování setlistů s koncerty budu implementovat až v následující fázi vývoje, proto je v aplikaci zatím uvedena informace „Setlist: Coming soon“.

Pro efektivní načítání seznamu událostí společně s přiřazenými skladbami jsem využil pokročilejší SQL dotaz s funkcí *json_agg*. Díky tomu nemusím pro každou zkoušku posílat další dotaz do databáze pro získání seznamu skladeb, ale všechna data dostanu v jedné odpovědi.

```
const getEvents = async (req, res) => {
  // ... validate
  let query = `
    SELECT e.event_id, e.title, e.type, e.date_time, e.place,
      -- Agregace skladeb do JSON pole přímo v SQL dotazu
      COALESCE((
        SELECT json_agg(
          json_build_object(
            'song_id', s.song_id,
            'title', s.title,
            'status', s.status
          )
        ) FILTER (WHERE s.song_id IS NOT NULL)
      FROM event_songs es
      JOIN songs s ON es.song_id = s.song_id
      WHERE es.event_id = e.event_id
      ), '[]') AS songs FROM events e WHERE e.band_id = $1
  `;
  // ...
};
```

Kód 8 - SQL dotaz pro získání událostí včetně skladeb

2.2.7 Správa úkolů (Todos)

Sekci s úkoly jsem rozdělil na dvě záložky – *My Todos*, kde uživatel vidí jen svoje věci, a *Band Todos*, kde je vidět přehled úkolů celé kapely. Při vývoji jsem musel vyřešit oprávnění, protože by nedávalo smysl, aby mohl kdokoli mazat cizí úkoly. Nastavil jsem to tedy tak, že úkol může označit za splněný, upravit nebo smazat pouze ten, kdo ho má přiřazený, kdo ho vytvořil nebo vedoucí kapely (Leader). Zároveň jsem vedoucímu dal možnost přiřazovat úkoly ostatním členům pomocí komponenty *StyledDropdown*. Běžný člen může úkol vytvořit pouze sám sobě. Pro formulář přidávání a úpravy úkolů jsem použil osvědčenou kombinaci knihoven *Formik* a *Yup* a pro výběr data splnění jsem použil knihovnu *react-native-community/datetimepicker*. Stejně jako u chatu jsem i zde použil tzv. optimistic update – když uživatel klikne na políčko úkolu, úkol se na displeji okamžitě odškrtně a přeškrtně, a až na pozadí se posílá dotaz na server. Pokud by server vrátil chybu, úkol se vrátí do původního stavu a aplikace zobrazí chybovou hlášku.

2.2.8 Sdílení hudebních nápadů

Tato sekce byla pro mě asi největší výzva ve druhém pololetí, protože pracuje s nahráváním zvuku přes mikrofon telefonu. Uživatel může vytvořit buď textový nápad (např. text písně), nebo audio nápad. Pro nahrávání zvuku jsem použil knihovnu *expo-audio*. Nejdříve se aplikace musí uživatele zeptat na povolení přístupu k mikrofonu pomocí funkce *AudioModule.requestRecordingPermissionsAsync()*. Samotné nahrávání probíhá do dočasného souboru v telefonu. Po dokončení nahrávání se soubor odešle na Firebase Storage přes podepsanou URL adresu, což funguje úplně stejně jako u ukládání skladeb v prvním pololetí. Během testování nahrávání jsem si uvědomil, že muzikant někdy potřebuje při nahrávání držet tempo. Vytvořil jsem proto komponentu metronomu, kterou jsem dal přímo do modalu pro nahrávání nápadu. Uživatel si tam může nastavit BPM a metronom si spustit. Nápady lze také označit hvězdičkou jako oblíbené a pak je podle toho filtrovat. V detailu nápadu pak jde upravovat tóninu, BPM a psát text. Pro zápis tabulatur jsem použil textové pole s neproporcionálním písmem (monospace), aby byly značky strun a pražců pod sebou. Pro usnadnění psaní jsem přidal tlačítka *+ Bass* a *+ Guitar*, která do textu vloží prázdné linky tabulatury.

U metronomu, který pomáhá držet rytmus při nahrávání nápadů, je zásadní přesné načasování. Obvyčejný JavaScript `setInterval` ale trpí na zpoždování kvůli zátěži procesoru nebo překreslování UI. Vyřešil jsem to vlastním časovou smyčkou přes `setTimeout`, který po každém tiknutí spočítá, o kolik milisekund se reálně zpozdil oproti očekávanému času další doby (*expectedTime*), a o tolik zkrátí délku příštího intervalu. Pokud aplikace na chvíli zamrzne (např. při přepnutí na pozadí), časovač se resetuje, aby se předešlo efektu "samopalu", kdy by se všechny zpožděné kliky přehrály naráz.

```
useEffect(() => {
  if (!isPlaying) {
    if (timeoutRef.current) clearTimeout(timeoutRef.current);
    setTick(false);
    return;
  }
  let expectedTime = Date.now();
  const playTick = () => {
    setTick(t => !t);
    Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy); // Hmatová
odezva (vibrace do ruky)
    clickPlayer.seekTo(0);
    clickPlayer.play();
    const currentBpm = bpmRef.current;
    const msPerBeat = 60000 / currentBpm;
    // Vypočítáme očekávaný čas další doby
    expectedTime += msPerBeat;
    const now = Date.now();
    let nextTime = expectedTime - now;
    // Pojistka pro případ, že se aplikace zasekla na pozadí, abychom se
vyhnuli rychlému přehrání nahromaděných tiků
    if (nextTime < 0) {
      expectedTime = now;
      nextTime = msPerBeat;
    }
    timeoutRef.current = setTimeout(playTick, nextTime);
  };
  playTick(); // První klik spustíme okamžitě
  return () => {
    if (timeoutRef.current) clearTimeout(timeoutRef.current);
  };
}, [isPlaying, clickPlayer]);
```

Kód 9 - Samoopravný metronom

2.2.9 Individuální záznam cvičení

Pro některé muzikanty je důležité sledovat, jak často cvičí doma. Vytvořil jsem proto sekci *My Practice*, kam si uživatelé mohou zapisovat odehraný čas. Záznam jde přidat buď ručně (zapsáním minut, data a poznámky do formuláře), nebo pomocí stopek přímo v aplikaci. Když uživatel otevře modal pro přidání cvičení, může kliknout na tlačítko *Start Timer*. V kódu toto mám vyřešené přes hook *useEffect* s časovým intervalem, který každou sekundu připočte jedničku do stavové proměnné. Jakmile uživatel cvičení dokončí a klikne na *Stop*, stopky se zastaví, sekundy se přepočítají na minuty (zaokrouhлено nahoru) a hodnota se automaticky vypíše do políčka pro minuty.

```
// Timer v komponentě cvičení
const [timerActive, setTimerActive] = useState(false);
const [timerSeconds, setTimerSeconds] = useState(0);

useEffect(() => {
  let interval = NodeJS.Timeout;
  if (timerActive) {
    interval = setInterval(() => {
      setTimerSeconds((prev) => prev + 1);
    }, 1000);
  }
  // Vyčištění intervalu při ukončení
  return () => clearInterval(interval);
}, [timerActive]);
```

Kód 10 - Časovač pro stopování délky cvičení

Na první pohled se tato funkce může zdát jako zbytečná, minimálně v tomto základním stavu. Momentálně bych s tím i souhlasil, ale mám v plánu ji výrazně rozšířit a dát jí důvod pro existenci. Chci provést gamifikaci cvičení – za cvičení bude uživatel například postupně odemykat úspěchy nebo dostávat odměny. Také bych mohl nechat vedoucího kapely zobrazit co kdo mezi zkouškami cvičil. Toto jsem ale už nemohl stihnout implementovat v časovém období školního projektu.

2.2.10 Setlisty

Správa setlistů je další část aplikace, která byla trochu náročnější, hlavně z hlediska UI. Setlist slouží k poskládání pořadí písniček na koncert. Nejdůležitější bylo vyřešit, jak skladby jednoduše přesouvat. Použil jsem k tomu knihovnu *react-native-draggable-flatlist*. Uživatel podrží prst na ikoně vedle názvu písničky a tažením ji přesune, kam potřebuje. Po puštění písničky se přepočítají indexy a nové pořadí se uloží do spojovací tabulky *setlists_songs*. Další věc, kterou jsem musel vyřešit, byl výpočet celkové délky setlistu. Délky jednotlivých písniček mám v databázi uložené jako typ interval. Napsal jsem funkci, která tyto intervaly převede na vteřiny, sečte je za všechny písničky v setlistu a výsledek pak naformátuje zpět do vzhledu minuty:sekundy. Při přidávání skladeb do setlistu se otevře modal, ve kterém lze vyhledávat a také filtrovat podle stavu skladby, tóniny a štítků, což kapele s hodně písničkami ušetří práci.

Tato funkce řeší problém s formátem času. PostgreSQL vrací délku skladeb z databáze jako objekt (např. { *minutes*: 4, *seconds*: 30 }) nebo jako řetězec. Abych mohl sečíst délku všech skladeb v setlistu a získat celkový čas setu, musel jsem napsat převodník, který jakýkoliv z těchto formátů převede na vteřiny, ty sečte pomocí metody *reduce()* a výsledek naformátuje pro zobrazení uživateli.

```
// Převod různých formátů délky na sekundy
const getLengthInSeconds = (length any) => {
  if (!length) return 0;
  if (typeof length === "object") {
    return (
      (parseInt(length.minutes) || 0) * 60 +
      (parseInt(length.seconds) || 0)
    );
  }
  if (typeof length === "string" && length.includes(":")) {
    const [m, s] = length.split(":");
    return (parseInt(m) || 0) * 60 + (parseInt(s) || 0);
  }
  return parseInt(length) || 0;
};

// Sečtení vteřin všech skladeb v setlistu
const totalLengthSeconds = songs.reduce(
  (sum, song) => sum + getLengthInSeconds(song.length),
  0,
);
const formattedTotalLength = formatTotalSeconds(totalLengthSeconds);
```

Kód 11 - Výpočet délky setlistu s převodem

2.2.11 Domovská obrazovka (Dashboard)

Úvodní stránka aplikace je to první, co uživatel po přihlášení uvidí, takže jsem chtěl, aby byla co nejužitečnější. Stránka zobrazuje rychlé odkazy na ostatní sekce, nejbližší události a seznam posledních aktivit v kapele. Abych mohl zobrazovat, co se v kapele zrovna děje, vytvořil jsem v databázi tabulku *activities*. Kdykoliv někdo v aplikaci udělá něco důležitého – například přidá novou skladbu, splní úkol nebo nahraje hudební nápad – backend automaticky zapíše záznam do této tabulky. Na frontendu pak načítám posledních 5 událostí. Aby to vypadalo lépe, napsal jsem v kódu podmínku, která kontroluje, kdo akci provedl. Pokud ji udělal aktuálně přihlášený uživatel, místo jeho jména se vypíše "You", což působí víc osobně. U událostí stahuji z databáze všechny akce a na frontendu je filtruji tak, aby se zobrazily jen tři nejbližší nadcházející akce (zkoušky nebo koncerty), u kterých je datum a čas v budoucnosti.

2.3 Popis pro uživatele

Tato kapitola slouží jako stručná uživatelská příručka, která provede nového uživatele základními funkcemi aplikace RehearsalHub.

2.3.1 Registrace a prvotní nastavení

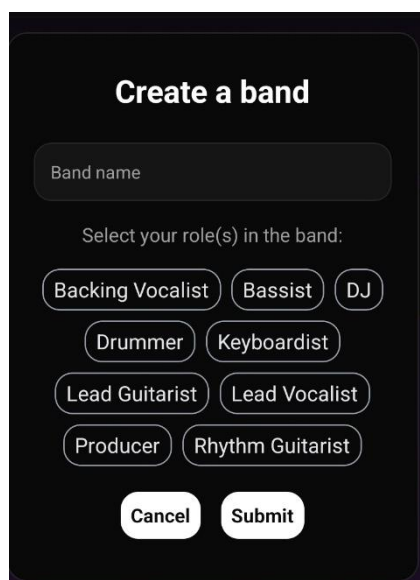
Při prvním spuštění aplikace se uživatel ocitne na přihlašovací obrazovce. Pokud ještě nemá účet, přepne se na záložku registrace. Zde má dvě možnosti:

- **Registrace emailem:** Uživatel vyplní uživatelské jméno, email a heslo. Po odeslání formuláře je nutné potvrdit registraci kliknutím na odkaz v ověřovacím emailu, jinak aplikace nepovolí přístup dále.
- **Přihlášení přes Google:** Rychlejší varianta, která nevyžaduje ověřování emailu a automaticky použije profilovou fotografii z Google účtu.

2.3.2 Vytvoření nebo připojení ke kapele

Po úspěšném přihlášení se uživatel ocitne na hlavní obrazovce. Pokud není členem žádné kapely, aplikace mu nabídne dvě možnosti:

- **Vytvořit kapelu (Create a band):** Uživatel zadá název nové kapely a zvolí si svou roli (např. Kytara, Zpěv). Automaticky se stává lídrem kapely (Leader), což mu dává právo později měnit název kapely nebo spravovat členy.
- **Připojit se ke kapele (Join a band):** Pokud již kapela existuje, uživatel musí zadat unikátní zvací kód (Invite Code). Tento kód najde lídr existující kapely v sekci *Band* na kartě s informacemi.



Obrázek 6 - Formulář pro vytvoření kapely. Zdroj: autor

V případě, že uživatel působí ve více kapelách, může se mezi nimi kdykoliv přepínat pomocí menu v pravém horním rohu nebo v nastavení profilu.

2.3.3 Nástěnka a přehled aktivit

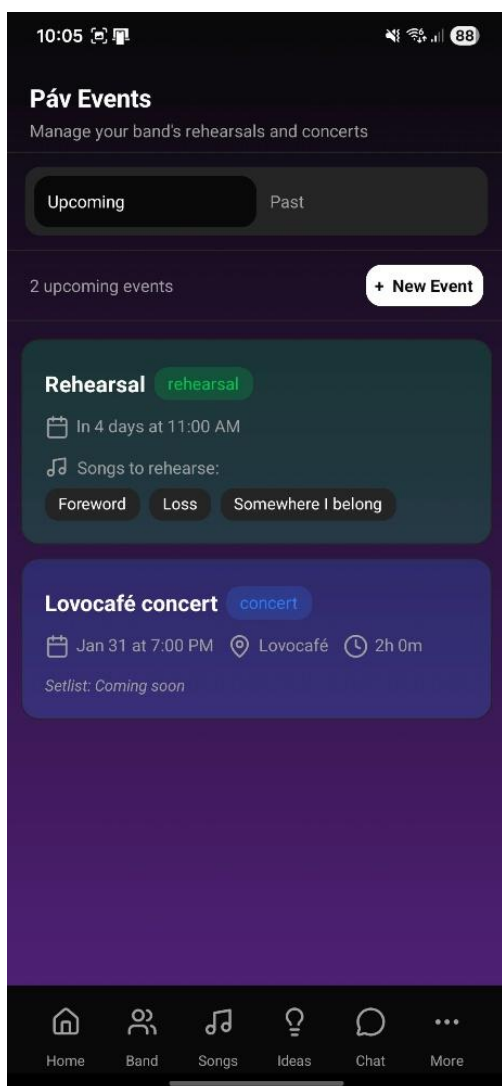
Po přihlášení a výběru kapely se uživateli zobrazí domovská obrazovka. V horní části jsou ikony pro rychlý přechod do nejdůležitějších sekcí. Uprostřed vidí seznam nejbližších plánovaných událostí a dole je feed posledních aktivit v kapele, kde je vidět, co ostatní členové zrovna dělali.

2.3.4 Plánování událostí

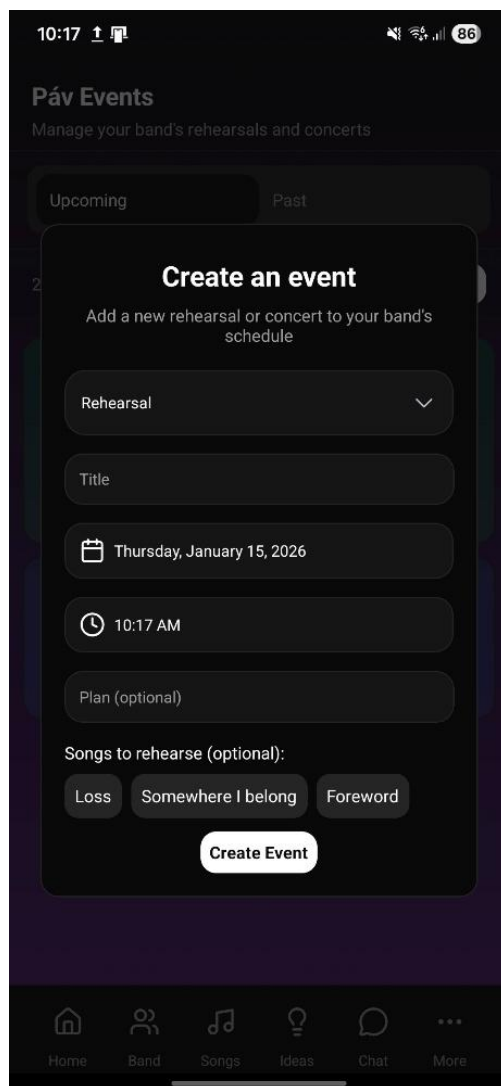
V sekci *Events* kapela spravuje svůj kalendář. Při vytváření nové události uživatel volí mezi dvěma typy:

- **Zkouška (Rehearsal):** Zde je možné ze seznamu vybrat konkrétní skladby, které se mají cvičit. Členové kapely tak předem ví, na co se připravit.
- **Koncert (Concert):** U koncertu se zadává místo konání a délka vystoupení.

Události jsou rozděleny na nadcházející a minulé, přičemž každá karta události přehledně zobrazuje datum, čas a v případě zkoušek i seznam skladeb.



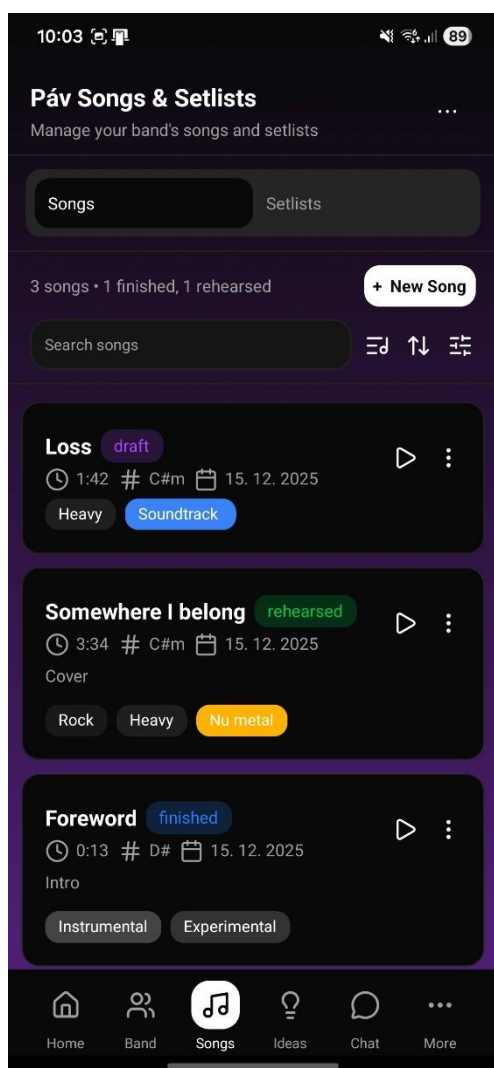
Obrázek 8 - Stránka událostí. Zdroj: autor



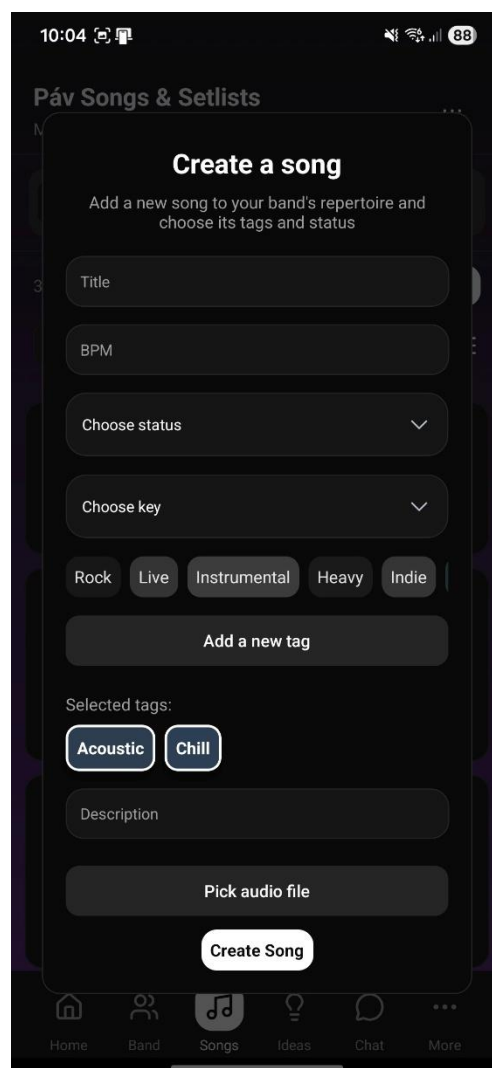
Obrázek 7 - Formulář pro přidání události. Zdroj: autor

2.3.5 Správa repertoáru a přehrávač

Sekce *Songs* slouží jako centrální úložiště. Uživatel může nahrát audio soubor (např. demo nahrávku ze zkoušky) a doplnit k němu metadata jako BPM, tóninu (Key) a stav rozpracování (Draft, Rehearsed, Finished). Pro lepší organizaci jsem implementoval systém štítků (Tags). Ke každé skladbě lze přiřadit až 4 barevné štítky, podle kterých lze následně seznam skladeb filtrovat. Přehrávání skladeb zajišťuje plovoucí přehrávač, který zůstává aktivní i při přechodu mezi stránkami aplikace, což umožňuje poslouchat nahrávku a zároveň například číst zprávy v chatu.



Obrázek 10 - Stránka úložiště skladeb. Zdroj: autor



Obrázek 9 - Formulář pro přidání skladby. Zdroj: autor

2.3.6 Komunikace

Pro domluvu slouží záložka *Chat*. Jedná se o společnou konverzaci pro všechny členy aktuálně vybrané kapely. Zprávy se řadí od nejnovějších a díky real-time propojení se zobrazují okamžitě všem připojeným členům bez nutnosti obnovovat stránku.

2.3.7 Správa úkolů

V sekci *Todos* kapela spravuje úkoly. Záložka *My Todos* zobrazuje úkoly přihlášeného uživatele, na záložce *Band Todos* jsou vidět úkoly všech členů. Kliknutím na políčko vedle úkolu se úkol označí jako splněný. Vedoucí kapely může při vytváření úkolu vybrat ze seznamu člena kapely a úkol mu přiřadit. (Obrázek 12 - Obrazovka úkolů)

2.3.8 Měření cvičení

Záložka *My Practice* slouží k zapisování času stráveného cvičením na nástroj. Uživatel může kliknout na tlačítko *+ Add Practice* a vyplnit údaje ručně, nebo použít časovač, který běží na displeji a po zastavení čas automaticky uloží.

2.3.9 Nahrávání a editace hudebních nápadů

V sekci *Ideas* mohou uživatelé sdílet hudební nápady. Při vytváření nového nápadu si uživatel zvolí, zda jde o text nebo o nahrávku. U nahrávky se zobrazí metronom a nahrávací tlačítko. V detailu uloženého nápadu lze doplnit tóninu, tempo, text a zapisovat tabulatury pomocí monospace editoru a pomocných tlačítek. (Obrázek 14 - Zápis tabulatury v detailu nápadu)

2.3.10 Tvorba setlistů

Setlisty se nacházejí na stránce *Songs* v záložce *setlists* a upravují se v detailu konkrétního setlistu. Kliknutím na *Add Song* se otevře modal pro vyhledání a přidání skladby. V samotném editoru setlistu může uživatel chytit skladbu za ikonu gripu na levé straně a tažením změnit její pozici v seznamu. Aplikace sama přepočítává celkovou délku setlistu. (Obrázek 15 - Úprava pořadí skladeb v setlistu)

Závěr

Hlavním cílem první fáze realizace mé ročníkové práce bylo navrhnout robustní architekturu a vytvořit funkční jádro mobilní aplikace RehearsalHub. Zaměřil jsem se především na to, aby aplikace stála na pevných základech – od návrhu relační databáze a backendového API až po zprovoznění klíčových mechanismů, jako je autentizace uživatelů a správa kapel. V tuto chvíli mohu konstatovat, že se mi podařilo vytvořit stabilní prostředí, ve kterém mohou uživatelé zakládat kapely, komunikovat v reálném čase, ukládat skladby a organizovat základní agendu. To tvoří nezbytný předpoklad pro implementaci pokročilejších hudebních funkcí ve druhém pololetí.

Za největší technický úspěch této etapy považuji implementaci skupinového chatu. I přes počáteční komplikace se mi podařilo zprovoznit komunikaci v reálném čase pomocí technologie Socket.IO a efektivně vyřešit načítání historie zpráv využitím kurzorové paginace. Další významnou výzvou, kterou jsem úspěšně překonal, byla integrace přihlašování přes Google účet. Zde jsem musel vyřešit problémy s nastavením přesměrovacích URI a omezeními v prostředí Expo Go. Z hlediska uživatelského rozhraní se podařilo implementovat globální kontexty pro přepínání kapel a změnu barevného schématu, což výrazně zvyšuje komfort používání aplikace.

Během vývoje jsem dospěl k poznání, že původní návrh vyžadoval v průběhu realizace revizi a doplnění. Ukázalo se, že pouhý seznam skladeb by byl pro uživatele nepřehledný, proto jsem operativně navrhl a implementoval systém filtrace pomocí štítků. Tato zkušenost mě upozornila na detaily, které jsem v prvotním plánování podcenil – zejména časovou náročnost ošetření chybových stavů (např. při výpadku připojení) a tvorbu přechodových animací. Tyto aspekty mi poskytly cennou lekci v oblasti projektového řízení a odhadu pracnosti, na což se v budoucnu zaměřím mnohem důsledněji.

Z osobního hlediska pro mě práce na tomto projektu představuje obrovský posun v chápání vývoje komplexního softwaru. Prakticky jsem si ověřil, že propojení teoretických znalostí o relačních databázích s reálným backendem v Node.js vyžaduje mnohem hlubší porozumění souvislostem, než se na první pohled zdá. Zdokonalil jsem se v práci s TypeScriptem, jehož typová kontrola mi ušetřila mnoho času při ladění chyb, a naučil jsem se pracovat s nástroji pro verzování kódu v týmovém kontextu, i když jsem na projektu pracoval převážně samostatně. Tato zkušenost mi potvrdila, že vývoj aplikace není jen o psaní kódu, ale především o schopnosti analyzovat problémy a hledat efektivní řešení.

V nadcházejícím pololetí budu pokračovat ve vývoji modulů, které dají aplikaci její finální podobu. Prioritou bude dokončení správy setlistů, vytvoření informativní domovské stránky (Dashboardu) a implementace systému pro správu úkolů a záznamů o cvičení. Zároveň se budu věnovat ladění uživatelského zážitku (UX), včetně přidání animací a vizuálních detailů, které aplikaci posunou na profesionálnější úroveň. Mým cílem je do konce školního roku odevzdat kompletní produkt, který bude připraven na reálné testování hudebními kapelami.

Seznam použitých zdrojů

1. **Leach, Arlo.** BandHelper: Band management software for iOS, Android, MAC and web. [Online] 20. Prosinec 2025. [Citace: 20. Prosinec 2025.] <https://www.bandhelper.com>.
2. **BandHelper.** Ukázka vzhledu aplikace BandHelper. [Online] [Citace: 20. Prosinec 2025.] https://www.bandhelper.com/images/content/home_book.jpg.
3. **Mattermost.** Mattermost | Collaboration Platform for Mission Critical Work. [Online] [Citace: 29. 12 2025.] <https://mattermost.com>.
4. **META PLATFORMS.** React Native · Learn once, write anywhere. [Online] 19. Prosinec 2025. <https://reactnative.dev/>.
5. **EXPO.** Expo: Make any app. Run it everywhere. [Online] 19. Prosinec 2025. <https://expo.dev/>.
6. **OPENJS FOUNDATION.** Node.js. [Online] 19. Prosinec 2025. <https://nodejs.org/>.
7. **NATIVEWIND.** NativeWind: A styling system for React Native. [Online] 19. Prosinec 2025. <https://www.nativewind.dev/>.
8. **POSTGRESQL GLOBAL DEVELOPMENT GROUP.** PostgreSQL: The World's Most Advanced Open Source Relational Database. [Online] 19. Prosinec 2025. <https://www.postgresql.org/about/>.
9. **Google LLC.** Firebase | Google's Mobile and Web App Development Platform. [Online] [Citace: 14. Leden 2026.] <https://firebase.google.com>.
10. **Figma, Inc.** Figma: The Collaborative Interface Design Tool. [Online] [Citace: 14. Leden 2026.] <https://www.figma.com>.
11. **GITHUB.** GitHub: Let's build from here. [Online] 19. Prosinec 2025. <https://github.com/about>.
12. **OPENAI.** ChatGPT. [Online] 19. Prosinec 2025. <https://openai.com/chatgpt>.

Seznam obrázků

Obrázek 1 - Ukázka vzhledu aplikace BandHelper. Zdroj: (2).....	9
Obrázek 2 - Mattermost. Zdroj: (3)	10
Obrázek 3 - Fotografie papíru s prvním návrhem projektu. Zdroj: autor.....	13
Obrázek 4 - ERD Databáze. Zdroj: autor	14
Obrázek 5 - Návrh designu aplikace. Zdroj: autor	15
Obrázek 6 - Formulář pro vytvoření kapely. Zdroj: autor.....	31
Obrázek 7 - Formulář pro přidání událostí. Zdroj: autor	32
Obrázek 8 - Stránka událostí. Zdroj: autor	32
Obrázek 9 - Formulář pro přidání skladby. Zdroj: autor.....	33
Obrázek 10 - Stránka úložiště skladeb. Zdroj: autor	33

Seznam kódů

Kód 1 - Definice autentizačního kontextu	17
Kód 2 - Definice kontextu kapely.....	18
Kód 3 - Načítání historie a kurzorová paginace	20
Kód 4 – Logika odeslání zprávy (Optimistic Update)	21
Kód 5 - Načtení barevného schéma z ThemeContextu	22
Kód 6 - nahrávání souboru na cloudové úložiště	23
Kód 7 - Definice kontextu přehrávače	24
Kód 8 - SQL dotaz pro získání událostí včetně skladeb.....	25
Kód 9 - Samoopravný metronom	27
Kód 10 - Časovač pro stopování délky cvičení.....	28
Kód 11 - Výpočet délky setlistu s převodem	29